

Unspecified Behaviour in C and C++

This article covers the various aspects of unspecified behaviour in the C and C++ programming languages. It is a sequel to earlier articles on undefined behaviour, published in *OSFY*.



Every programmer aims to have error-free code during compilation. Sometimes you end up with code devoid of any syntax errors, but might have a few runtime errors. Even if all such errors are rectified and the program is shipped to the end users, there could be potential bugs ready to be triggered at any instant. There are numerous reasons for such bugs, including problems like unrealistic time schedules, incorrect designs, etc. One reason for bugs in the software might be the presence of code that exhibits undefined behaviour. A programmer should also know about unspecified behaviour to avoid the presence of non-standardised code in software. Unspecified behaviour in a particular programming language refers to behaviour that may vary depending on the different implementations of that programming language. In this article we will discuss unspecified behaviour in C and C++. We will also have a short discussion about implementation-defined behaviour in C and C++, which is slightly different from unspecified behaviour.

But before proceeding any further, let us try to distinguish between unspecified behaviour and undefined behaviour. The latter has already been covered in earlier issues of *OSFY*. The C11 standard defines undefined behaviour as "...behaviour, upon use of a non-portable or erroneous program construct or of erroneous data, for which this International Standard imposes no requirements." This definition itself is a mild warning against the use of constructs which might lead to undefined behaviour in a program. In general, we must refrain from using code that will cause undefined behaviour because

such code may lead to syntax errors, runtime errors, or luckily even the expected behaviour from the point of view of a programmer. So, it is absolutely dangerous to have code with undefined behaviour in your programs. Integer division by zero, modifying an object more than once before reaching a sequence point, etc, are examples of undefined behaviour.

The C11 standard defines unspecified behaviour as "...behaviour where this International Standard provides two or more possibilities and imposes no further requirements on which is chosen in any instance." So, unlike code exhibiting undefined behaviour which should be avoided in a program at any cost, the programmer has the freedom to use code exhibiting unspecified behaviour. But the output of the code with unspecified behaviour may not be the same for all the compilers and all the systems. As we proceed further, we will see examples wherein the same program in the same system gives different outputs when compiled with different compilers. So, a programmer writing code for a single system and a single compiler need not worry about unspecified behaviour. But a programmer writing code with portability as an intention should worry about both undefined behaviour and unspecified behaviour.

Unlike code exhibiting undefined behaviour, code that exhibits unspecified behaviour is often included in many C and C++ programs. You will not get any further guidelines regarding the behaviour of code exhibiting unspecified behaviour either in the programming language standard or in the compiler implementation manual. Still, code exhibiting